Modular, self-evolving agent runtimes

Hugo O'Connor · `codeberg.org/anuna` · 2026-04-28 *Position paper · CC-BY-4.0*

Abstract

Most LLM-agent frameworks today embed assumptions about what models can or cannot do — planner modules, prompt-template engines, output parsers, hand-written tool-dispatch logic. Such assumptions age badly under Sutton's bitter lesson: as model capability climbs, scaffolding becomes obsolete and the framework's value collapses into migration burden. We argue that agent runtimes should commit to *operational scaffolding* — supervision, identity, audit, capability routing — and treat every other layer as part of a *modular, self-evolving runtime* the user can redefine in flight and the agent itself can extend under safety gates. Three principles follow: every layer redefinable at runtime without restart, the runtime artifact is the heap, and safety comes from invariants, not pipelines. The harness is not a finished product but accumulates over time, as the agent/user team meets the real requirements of each task. We offer **anuna-imago**, a ~4100-LOC SBCL Common Lisp harness, as an existence proof. We acknowledge the wager: if model capability plateaus, the prescriptive frameworks were correct to bake in scaffolding.

1. Introduction

What should an LLM-agent runtime commit to? The mainstream answer over the past three years has been to give the developer a planner, a memory module, a tool selector, a prompt-template engine, and a curated dispatch loop. These abstractions answer the question "*what does the model need help doing?*" — and in answering it, they commit the framework to a specific theory of contemporary model limitation. The thesis of this paper is that such commitments are wagers against capability scaling, and they have been losing.

Concrete examples — without naming particular projects, since the structural argument is independent of any one of them — are easy to find. Frameworks that shipped explicit planner modules in 2022 found that 2023's chain-of-thought tool-use loops did the same work in less code. Frameworks that shipped regex or grammar-based output parsers in 2023 found that 2024's structured-output modes obsoleted them in months. Frameworks that built elaborate prompt-template engines for fragile instruction-following found that the templating value shrank as instruction-following improved. In every case the framework's abstractions encoded a model limitation, the limitation lifted, and the abstraction became dead weight or migration debt.

The alternative we propose is to refuse, at the framework layer, to encode assumptions about model capability. Commit only to *operational scaffolding* — supervision, identity, audit, capability routing, image distribution, runtime safety invariants. These are about how the agent process *exists and is governed*, not about what work it does. They are stable across model generations because they are not predictions about model behaviour at all.

This stance has consequences. Three principles structure them: every layer must be redefinable at runtime without restart, the runtime artifact must be the heap, and safety must come from invariants rather than from prescriptive pipelines. A fourth principle puts these in motion: the harness accumulates as the agent and user team meets the real requirements of the task.

Contributions. (1) An argument that the bitter lesson applies to agent-runtime framework design, with three concrete examples of capability-tracking abstractions that have already aged out (§2). (2) Four principles that characterise a modular, self-evolving runtime: live redefinition, image-as-artifact, invariants over pipelines, and harness accumulation (§3). (3) Four minimum requirements that operationalise those principles into testable criteria a candidate runtime can be checked against (§4). (4) anuna-imago, an SBCL Common Lisp harness of approximately 4100 lines, offered as an existence proof (§5). (5) A statement of the wager: this position loses if model capability plateaus (§7).

2. The bitter lesson, applied to runtimes

Sutton's *bitter lesson* [Sutton 2019] holds that across game-playing, speech, vision, and translation, "general methods that leverage computation are ultimately the most effective, and by a large margin." The history of each subfield has been one of researchers building elaborate domain heuristics, only to be overtaken by simpler methods that scaled with compute. Sutton's argument concerns *research methods* — what to spend a PhD or a lab budget on. We argue the lesson transfers cleanly to *engineering frameworks*: abstractions that compensate for current model limitations are themselves hand-engineering, and they age out in the same way and for the same reason.

Three concrete examples make the transfer concrete. **Explicit planner modules** were a major offering of 2022-era agent frameworks; they encoded a theory that LLMs could not decompose tasks without help. Tool-using chain-of-thought loops have since done the same work directly, and the explicit-planner modules now exist mainly to be skipped. **Output parsers** — regex, grammar, BNF — were necessary infrastructure when models would not reliably produce valid JSON. Within a year, structured-output and tool-calling modes made the parsers vestigial; the libraries are now legacy maintenance for code paths most users no longer enter. **Prompt-templating engines** built around the assumption that instruction-following was fragile lost most of their value as instruction-following improved. The abstractions remain in framework code; the user no longer needs them.

A natural counterargument is that *every* framework needs *some* opinion — the choice is not between opinionated and unopinionated but between which opinions to hold. We accept the framing and refine the claim: framework opinions about *operational concerns* (process supervision, audit, distribution, identity, capability routing) are stable across model generations because they are not predictions about model behaviour at all. Framework opinions about *capability concerns* (what the model needs help doing) are unstable because they are exactly such predictions. The advice is not "have no opinions" but "hold the opinions that don't depend on contingent facts about today's models."

The migration cost of capability-tracking opinions is underdiscussed. Each time a model generation obsoletes a framework abstraction, engineering teams choose between rewriting against the new framework idiom (paying migration debt) or carrying dead code paths (paying maintenance debt). Neither outcome rewards the original choice. A framework that confines itself to operational concerns *cannot owe* this migration debt, because the operations themselves did not change.

3. The modular, self-evolving runtime

We use *modular, self-evolving runtime* to name an agent harness in which (a) every layer is replaceable or redefinable while the system runs, and (b) the agent itself can extend or rewrite its own runtime under safety gates. The framework provides infrastructure for the choices that don't depend on model capability; the user — or, where invariants permit, the agent — provides the choices that do. Three principles make the term concrete.

3.1 Every layer redefinable at runtime without restart

The framework treats methods as dispatch points, not call sites. Tool selection, hook handling, provider streaming, prompt assembly — each is a named operation whose implementation can be replaced while the system runs. The lineage runs through McCarthy's metacircular `eval` [McCarthy 1960] and its descendants in Lisp and Smalltalk: the language can interpret and modify its own representation. When a built-in in the framework turns out to encode an obsolete assumption, the user redefines it at the live REPL instead of filing a framework migration ticket. The framework cannot bake "the right way to do X" into a closed implementation; instead it exposes the seam where X is dispatched and offers a default. This is more demanding than configurability — configuration assumes the framework anticipated the axes of variation. Late binding does not.

3.2 The runtime artifact is the heap

Distribution is image distribution. The deployed binary is a frozen heap, including any patches the user applied at the live REPL before the snapshot was taken. There is no separate deployment pipeline whose assumptions might themselves age out — there is no pipeline, only the heap. A consequence is that the redefined-in-flight semantics survives deployment: the binary behaves exactly as the running image did at the moment of save. SBCL's `save-lisp-and-die` produces a single-file executable in this style [Steele & Gabriel 1996], descended from the Lisp-Machine and Smalltalk image traditions [Goldberg & Robson 1983]. The implication for framework design is that any tool the framework offers must be safe to freeze mid-flight: open files, credentials, network handles must either be cleaned before save or tolerate restoration on resume.

3.3 Safety from invariants, not pipelines

Safety is not a sequence of checks the framework runs in a fixed order; it is a set of statements about the system that must always hold. The framework gates on whether invariants are violated, but does not own the invariants themselves. The user authors them — in this implementation, in defeasible logic — and the framework consults them at the relevant points. The practical consequence is that when the model gains a new capability, the invariants don't need to change. A pipeline-based safety story, by contrast, gets longer with every new capability surface: a new tool means a new validator, a new orchestration step, a new place to forget a check. An invariant-based story stays the same size: the new capability either does or does not violate the existing rules. anuna-imago's self-modification port queries a Spindle defeasible-logic theory at every `harness-eval` call; the theory is the entire safety contract for that surface, and the framework's role is reduced to "ask the reasoner, veto on positive verdict."

3.4 The harness accumulates

Together, the three principles describe a harness that does not ship as a finished product but accumulates. The minimal commitment — supervision, identity, audit, capability routing — is the starting point, not the end. Beyond that, the harness grows in response to the requirements the agent and user team actually encounter in service of the task: a tool defined when a workflow demands it, a hook installed when behaviour needs to be observed, a method redefined when a default proves wrong. The runtime is the cumulative product of contact with real requirements, not the prediction of imagined ones. This is the inversion the bitter lesson asks for: instead of front-loading abstractions to anticipate model capability, the framework lets the abstractions emerge from use, and remain only as long as they remain useful.

4. Minimum requirements

A runtime is *modular and self-evolving* if and only if it satisfies the four requirements below. Each follows from one of the principles in §3, but is stated here as a testable criterion a candidate runtime can be evaluated against. They are minimal: a runtime that fails any one of them sacrifices the corresponding principle. They are not exhaustive: a candidate may meet all four and still be inadequate for other reasons (poor concurrency model, weak debugging, hostile build system). The requirements address modularity and self-evolution specifically.

R1 (Live redefinition). Every layer the user might wish to replace — tool dispatch, hook execution, provider streaming, prompt assembly, the turn loop — is reached through late binding. A replacement takes effect on the next call without restart. Configurability does not satisfy R1; configuration assumes the framework anticipated the axes of variation, while late binding does not.

R2 (Heap-as-artifact). The deployed runtime is a snapshot of the in-memory state, capturing any redefinitions applied before the snapshot. A source tree plus build step does not satisfy R2: it forces every live redefinition to be reified as a code edit before deployment, which breaks the affordance.

R3 (Invariant-based safety). Safety policy is expressible as statements over the runtime's actions, separable from the policy enforcer. The user authors policy without modifying the framework. The enforcer's role is reduced to consulting the policy at decision points and acting on its verdict.

R4 (Accumulation surface). Adding a tool, hook, or method does not require modifying the framework. The user-callable register/install/define operations take effect immediately (per R1) and persist across image saves (per R2).

A candidate runtime that satisfies R1–R4 is a *modular, self-evolving* runtime in the sense developed here. We use this definition in §5 to evaluate anuna-imago against it.

5. anuna-imago as existence proof

We have built a runtime that follows the three principles, called **anuna-imago**. The implementation is approximately 4100 lines of Common Lisp on SBCL with a further 3340 lines of tests; the harness is small enough to read in an afternoon, and small enough that no part of it is load-bearing in the sense that the user cannot replace it. We offer the artifact as a demonstration that the principles cohere into a working system, not as a product pitch — the choice of substrate (SBCL Common Lisp) carries real costs that we acknowledge explicitly below.

5.1 A worked redefinition (R1, R2, R4)

Any method can be redefined at the live REPL, and the redefinition survives an image save. The following transcript builds an agent against a stub provider, asks it, redefines the provider's `stream!` method, asks again (new behaviour, same process, no restart), and saves the heap as an executable binary. Running the binary from the shell produces output

consistent with the redefined method:

```
(defparameter *agent* (build-echo-agent))
(spawn-agent! *sup* *agent*)
(getf (ask-agent *agent* "hi") :text)        ; => "echo: hi"

(defmethod provider-stream! ((p stub-provider) agent message)
  (declare (ignore agent))
  (let* ((c (if (listp message) (getf message :content) message))
         (cell (cons nil (list (list :text (format nil "shouted: ~A!"
                                                   (string-upcase c)))))))
    (lambda () (pop (cdr cell)))))

(getf (ask-agent *agent* "hi") :text)        ; => "shouted: HI!"

(save-image! "shouty-agent" :toplevel 'agent-main)
```

```
$ ./shouty-agent --echo "hello"
shouted: HELLO!
```

No framework migration ticket; no rebuild step. The patch survives because the binary *is* the heap.

5.2 Safety from invariants in practice (R3)

The self-modification port instantiates Principle 3. A `harness-eval` tool lets the agent submit Common Lisp source forms for evaluation; before evaluation, three layers gate the call: a pre-filter denylist for obvious structural violations, a defeasible-logic reasoner querying a Spindle theory of floor invariants, and a handler that runs the form under timeout with a rollback register for any methods it redefines. A published log of six goal-driven runs with GLM 5.1 records the agent self-redefining 18 symbols across 6 turns to add persistent memory to itself, with 3 vetoes from the floor invariants intercepting attempts to redefine the safety surface. The invariants are short, queryable, and authored once.

5.3 Limitations

Common Lisp on SBCL is a small ecosystem with a sharp learning curve and genuine hiring difficulty. Teams committed to Python or TypeScript ergonomics will not adopt anuna-imago verbatim. Our argument is structural: *some* substrate that supports these principles must exist, and substrates that do not — including the typical Python-based agent stack — pay back the limitation as migration cost over time. anuna-imago demonstrates that the position is realisable; it does not claim to be the realisation everyone should choose.

6. Related stances

The position has antecedents and convergent neighbours.

The Lisp / Smalltalk image lineage [McCarthy 1960; Goldberg & Robson 1983]. The substrate spirit we adopt — `eval` / `apply` self-reference, late binding, image-as-artifact — is the inheritance of fifty years of metacircular language design. The lineage applied these properties to *integrated development*: Lisp Machines and Smalltalk-80 were workstations where the developer and the program lived in the same heap. Our claim is that the spirit transfers to LLM-agent runtimes, and that the bitter lesson makes the transfer newly important — not because agent runtimes are a new application domain, but because they are the first application domain where the *user of the runtime* (the LLM) has capability that climbs faster than the framework's release cycle.

Anthropic's "Building Effective Agents" [Anthropic 2024]. The post argues empirically that simpler agent loops — plain tool-use, no orchestration framework — often outperform complex multi-agent architectures. We reach the same practical conclusion from a different argument: the bitter lesson predicts that orchestration abstractions will pay obsolescence cost over time.

The Erlang/OTP supervision tradition [Armstrong 2003]. anuna-imago's `make-supervisor` is a direct CLOS port of Armstrong's one-for-one supervisor, and its mailbox abstraction follows the same letterbox-with-pid shape as Erlang processes. The contribution we claim is *not* the supervisor — Armstrong's design is fully borrowed and unmodified. The

contribution is to limit framework scope to such established prior art and push every other concern to user code. Operational scaffolding is not a new research problem; what is new is committing to it as the *only* framework concern.

7. Conclusion

Agent runtimes should commit to operational scaffolding only — supervision, identity, audit, capability routing — and treat every other layer as part of a modular, self-evolving runtime that the user, or the agent itself, can redefine in flight. Three principles implement the position: live redefinition, image-as-artifact, invariants not pipelines. anuna-imago, ~4100 LOC of Common Lisp, demonstrates that the position is realisable. The wager is plain: if model capability plateaus in the next several years, prescriptive frameworks were correct to bake in scaffolding and we paid for under-engineering. If capability continues climbing, prescriptive frameworks pay migration debt forever and the modular-runtime position wins by attrition. Either way, the harness is not built before the task arrives — it accumulates as the agent/user team learns what the task actually requires. The artifact is at `codeberg.org/anuna/imago`. Take it apart, and grow what you need.

References

Anthropic (2024). *Building Effective Agents.* Engineering blog post, `anthropic.com/engineering/building-effective-agents`.

Armstrong, J. (2003). *Making reliable distributed systems in the presence of software errors.* PhD thesis, KTH.

Goldberg, A. & Robson, D. (1983). *Smalltalk-80: The Language and its Implementation.* Addison-Wesley.

McCarthy, J. (1960). Recursive Functions of Symbolic Expressions and Their Computation by Machine, Part I. *Communications of the ACM* 3(4):184–195.

Steele, G. & Gabriel, R. (1996). The evolution of Lisp. *History of Programming Languages II.* ACM Press.

Sutton, R. (2019). *The Bitter Lesson.* `incompleteideas.net/IncIdeas/BitterLesson.html`.